

MATERIA: DESARROLLO WEB BACKEND

GUÍA DE LABORATORIO N° 11

Seguridad de la API con Laravel Sanctum

Material de Práctica de Laboratorio

CONTENIDO DE LA GUÍA

- 1. Autenticación de APIs con tokens**
 - Tokens de acceso personal vs. sesiones SPA
- 2. Activación de Laravel Sanctum**
 - El trait HasApiTokens en el modelo User
- 3. Pertenencia del recurso**
 - Relacionar cada tarea con su usuario (refactor)
- 4. Endpoints de autenticación**
 - register, login, logout y user con tokens
- 5. Rutas protegidas con auth:sanctum**
- 6. Control de acceso por propietario**
 - Restricción 403 a recursos ajenos
- 7. Habilidades (abilities) y revocación**
- 8. Pruebas del flujo completo con Postman**

OBJETIVO DE LA GUÍA

Asegurar la API REST construida en la GL10 implementando autenticación basada en tokens con Laravel Sanctum. Al finalizar, el estudiante será capaz de activar Sanctum, emitir y revocar tokens de acceso personal, relacionar cada recurso con su usuario, proteger rutas con el middleware auth:sanctum, restringir el acceso por propietario (403), asignar habilidades a los tokens y verificar todo el flujo —incluido el rechazo 401— con Postman y cURL.

Datos de la asignatura	
Asignatura	INF560 – Desarrollo Web Backend
Carrera	Ingeniería Informática · Facultad de Ciencias Puras
Universidad	Universidad Autónoma Tomás Frías (UATF) – Potosí
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Stack	Laravel 13 · PHP 8.3+ · PostgreSQL · Laravel Sanctum
Duración estimada	2.5 a 3 horas (1 sesión de laboratorio)

1. Objetivos, Prerrequisitos y Resultados

Objetivos específicos

- Diferenciar la autenticación basada en tokens de la basada en sesiones, e identificar cuándo conviene cada una.
- Activar Laravel Sanctum en el proyecto y entender el modelo de tokens de acceso personal.
- Dar pertenencia al recurso: relacionar cada tarea con un usuario y refactorizar el controlador en consecuencia.
- Construir endpoints de registro, inicio y cierre de sesión que emitan y revoken tokens.
- Proteger rutas con el middleware auth:sanctum y restringir el acceso a recursos por propietario (403).
- Asignar y comprobar habilidades (abilities) sobre los tokens, y probar todo el flujo con Postman y cURL (incluido el 401).

Prerrequisitos

- Haber completado la GL10: el proyecto api-tareas con la API REST pública funcionando (tag v1.0.0).
- Tener instalados PHP 8.3+, Composer y PostgreSQL, con el proyecto api-tareas operativo.
- Conocer los códigos de estado 401 (no autenticado) y 403 (sin permiso).
- Tener Postman (o cURL) para consumir la API.

CONTINUACIÓN DE LA GL10

Esta guía toma el proyecto api-tareas tal como quedó en la GL10 (una API pública) y le añade una capa de seguridad con Laravel Sanctum. Trabajaremos sobre el mismo repositorio; al final lo etiquetaremos como v2.0.0.

2. Marco Teórico

2.1 El problema: una API pública es insegura

En la GL10 cualquiera con la URL podía crear, leer y borrar tareas. Eso es inaceptable en una aplicación real: necesitamos saber quién hace cada petición y limitar lo que puede hacer. La solución es la autenticación, que en una API no puede depender de cookies de sesión como en una app web tradicional.

2.2 Tokens vs. sesiones: el esquema Bearer

En una app web, el navegador guarda una cookie de sesión y la envía automáticamente. Pero un cliente que no es navegador (una app móvil, otro servidor) necesita un mecanismo distinto: enviar en cada petición una credencial portable. Esa credencial es el token, y se transmite en la cabecera HTTP Authorization con el esquema Bearer.

TOKEN DE ACCESO PERSONAL (PERSONAL ACCESS TOKEN)

Es una cadena aleatoria que identifica a un usuario ante la API. Sanctum guarda en la base de datos solo el hash SHA-256 del token, nunca el texto plano. El cliente lo envía en cada petición y el servidor lo verifica. Pueden emitirse varios tokens por usuario (uno por dispositivo) y revocarse de forma individual.

2.3 Laravel Sanctum: dos modos, un solo enfoque

Laravel Sanctum es el paquete oficial y ligero de Laravel para autenticar APIs. Resuelve dos problemas distintos:

Modo	Para qué sirve	Credencial
Tokens de API (Personal Access Tokens)	Apps móviles, clientes de terceros y APIs simples. El cliente envía un token en la cabecera <code>Authorization: Bearer</code>	Token (texto)
Autenticación de SPA	Frontends de primera parte (React/Vue) en el mismo dominio. Usa cookies de sesión y CSRF; no emite tokens.	Cookie de sesión

ENFOQUE DE ESTA GUÍA

Trabajaremos con el modo de tokens de acceso personal, estándar para una API REST consumida por apps móviles o clientes externos. El modo SPA queda como lectura complementaria.

2.4 Piezas clave de Sanctum

HasApiTokens — Trait que se agrega al modelo User. Aporta el método `createToken()` y la relación `tokens()`.

`createToken(nombre, [abilities])` — Genera un token. Devuelve un objeto `NewAccessToken`; su propiedad `plainTextToken` es el único momento en que se ve el token en texto plano.

`auth:sanctum` — Middleware que protege las rutas: rechaza con 401 cualquier petición sin un token válido.

`currentAccessToken()` — Devuelve el token con el que se autenticó la petición actual; se usa para revocarlo en el logout.

`tokenCan(habilidad)` — Comprueba si el token actual tiene una habilidad concreta antes de permitir una acción.

2.5 El ciclo de vida de la autenticación

1. El usuario se registra o inicia sesión enviando sus credenciales (POST `/api/login`).
2. El servidor valida las credenciales y, si son correctas, llama a `createToken()` y devuelve el `plainTextToken`.
3. El cliente guarda el token y lo envía en cada petición posterior: `Authorization: Bearer {token}`.
4. El middleware `auth:sanctum` identifica al usuario a partir del token; si falta o es inválido, responde 401.
5. Al cerrar sesión, el servidor borra el token actual (revocación), invalidándolo para futuras peticiones.

3. Desarrollo de la Práctica

Continúa sobre el proyecto api-tareas de la GL10. Cada actividad incluye pasos numerados, código con su ruta y un **Checkpoint** de verificación.

Actividad 1 · Activar Sanctum en el modelo User

Sanctum ya quedó instalado en la GL10 (el comando `install:api` creó la tabla `personal_access_tokens`). Solo falta habilitarlo en el modelo User.

1. Agrega el trait `HasApiTokens` al modelo User:

app/Models/User.php

```
<?php
namespace App\Models;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, Notifiable;

    protected $fillable = ['name', 'email', 'password'];
    protected $hidden = ['password', 'remember_token'];

    protected function casts(): array
    {
        return ['email_verified_at' => 'datetime', 'password' => 'hashed'];
    }
}
```

NO OLVIDES EL TRAIT

Sin `HasApiTokens`, el método `$user->createToken()` no existe («Call to undefined method»). El cast `password => hashed` cifra la contraseña automáticamente al asignarla.

CHECKPOINT

El modelo User usa el trait `HasApiTokens` y la tabla `personal_access_tokens` existe.

Actividad 2 · Dar pertenencia al recurso (refactor)

En la GL10 las tareas no tenían dueño. Ahora que habrá usuarios autenticados, cada tarea debe pertenecer a uno. Este es el cambio central de la guía: pasamos de un recurso público a uno privado por usuario.

1. Crea una migración para agregar la clave foránea `user_id` a la tabla `tareas`:

Terminal

```
php artisan make:migration add_user_id_to_tareas_table
```

database/migrations/xxxx_add_user_id_to_tareas_table.php

```
public function up(): void
{
    Schema::table('tareas', function (Blueprint $table) {
        $table->foreignId('user_id')
            ->constrained()
            ->cascadeOnDelete();
    });
}
```

DATOS EXISTENTES SIN DUEÑO

Como las tareas creadas en la GL10 no tienen `user_id`, añadir una columna obligatoria fallaría sobre esos registros. Al ser un proyecto de aprendizaje, regenera la base con `php artisan migrate:fresh` (esto borra los datos previos, lo cual es aceptable aquí).

2. Define las relaciones en ambos modelos y regenera la base:**app/Models/Tarea.php (añadir)**

```
use Illuminate\Database\Eloquent\Relations\BelongsTo;

public function user(): BelongsTo
{
    return $this->belongsTo(User::class);
}
```

app/Models/User.php (añadir)

```
public function tareas()
{
    return $this->hasMany(\App\Models\Tarea::class);
}
```

Terminal

```
php artisan migrate:fresh
```

CHECKPOINT

La tabla `tareas` tiene la columna `user_id` como clave foránea. Las relaciones `Tarea→User` (`belongsTo`) y `User→Tareas` (`hasMany`) están definidas.

Actividad 3 · Endpoints de autenticación

1. Crea un controlador para la autenticación:

Terminal

```
php artisan make:controller Api/AuthController
```

2. Implementa register, login, logout y user. Cada token se nombra con el dispositivo que lo solicita:

app/Http/Controllers/Api/AuthController.php

```
<?php
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

class AuthController extends Controller
{
    public function register(Request $request)
    {
        $datos = $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users,email',
            'password' => 'required|string|min:8|confirmed',
            'device_name' => 'required|string',
        ]);

        $user = User::create([
            'name' => $datos['name'],
            'email' => $datos['email'],
            'password' => $datos['password'],
        ]);

        $token = $user->createToken($datos['device_name'])->plainTextToken;

        return response()->json(['user' => $user, 'token' => $token], 201);
    }

    public function login(Request $request)
    {
        $datos = $request->validate([
            'email' => 'required|email',
            'password' => 'required|string',
            'device_name' => 'required|string',
        ]);

        $user = User::where('email', $datos['email'])->first();

        if (!$user || !Hash::check($datos['password'], $user->password)) {
            throw ValidationException::withMessages([
                'email' => ['Las credenciales son incorrectas.'],
            ]);
        }

        $token = $user->createToken($datos['device_name'])->plainTextToken;

        return response()->json(['user' => $user, 'token' => $token]);
    }

    public function logout(Request $request)
    {
        $request->user()->currentAccessToken()->delete();
        return response()->json(['message' => 'Sesión cerrada.']);
    }
}
```

```

    }

    public function user(Request $request)
    {
        return response()->json($request->user());
    }
}

```

CONFIRMED Y DEVICE_NAME

La regla password|confirmed exige que el cliente envíe también password_confirmation. El campo device_name (p. ej. «postman») permite emitir un token por dispositivo y revocarlos por separado.

CHECKPOINT

El AuthController tiene los cuatro métodos. register y login devuelven un token en plainTextToken; logout revoca el token actual.

Actividad 4 · Proteger las rutas con auth:sanctum

1. Reescribe routes/api.php: las rutas de autenticación son públicas; el resto exige token. Observa que las rutas de tareas se mueven dentro del grupo protegido:

routes/api.php

```

<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\TareaController;

// Rutas publicas
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

// Rutas protegidas (requieren token valido)
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user', [AuthController::class, 'user']);
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::apiResource('/tareas', TareaController::class);
});

```

2. Verifica el mapeo y el middleware:

Terminal

```
php artisan route:list --path=api
```

CHECKPOINT

Las rutas de register y login son públicas; las de user, logout y tareas aparecen con el middleware auth:sanctum.

Actividad 5 · Acotar el recurso por propietario

Ahora refactorizamos el TareaController de la GL10 para que cada usuario solo vea y manipule sus propias tareas. La clave: toda consulta parte de `$request->user()->tareas()`.

1. Reemplaza el controlador por esta versión con control de propiedad:

`app/Http/Controllers/Api/TareaController.php`

```
<?php
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Requests\TareaRequest;
use App\Http\Resources\TareaResource;
use App\Models\Tarea;
use Illuminate\Http\Request;

class TareaController extends Controller
{
    public function index(Request $request)
    {
        return TareaResource::collection(
            $request->user()->tareas()->latest()->paginate(10)
        );
    }

    public function store(TareaRequest $request)
    {
        $tarea = $request->user()->tareas()->create($request->validated());
        return (new TareaResource($tarea))
            ->response()->setStatusCode(201);
    }

    public function show(Request $request, Tarea $tarea)
    {
        $this->autorizarDueno($request, $tarea);
        return new TareaResource($tarea);
    }

    public function update(TareaRequest $request, Tarea $tarea)
    {
        $this->autorizarDueno($request, $tarea);
        $tarea->update($request->validated());
        return new TareaResource($tarea);
    }

    public function destroy(Request $request, Tarea $tarea)
    {
        $this->autorizarDueno($request, $tarea);
        $tarea->delete();
        return response()->json(null, 204);
    }

    private function autorizarDueno(Request $request, Tarea $tarea): void
    {
        abort_unless($tarea->user_id === $request->user()->id, 403);
    }
}
```

DOS MECANISMOS DE PROTECCIÓN

`index` y `store` nacen del usuario autenticado (`$request->user()->tareas()`), así que ya están acotados. Para `show`, `update` y `destroy`, el route-model binding inyecta la tarea por su `id`; `abort_unless` verifica que pertenezca al usuario y, si no, responde 403 Forbidden. En un proyecto mayor esta lógica se trasladaría a una Policy (TareaPolicy).

CHECKPOINT

Crear una tarea la asocia al usuario autenticado; intentar ver/editar/borrar una tarea de otro usuario devuelve 403.

Actividad 6 · Habilidades y revocación de tokens

Las habilidades (abilities) son permisos finos que viajan dentro del token, similares a los «scopes» de OAuth. Permiten, por ejemplo, emitir un token de solo lectura.

1. Emite un token con habilidades pasando un segundo argumento a `createToken()`:

Ejemplo (variación del login)

```
$token = $user->createToken(
    $datos['device_name'],
    ['tarejas:leer', 'tarejas:crear'] // habilidades del token
)->plainTextToken;
```

2. Comprueba la habilidad antes de actuar dentro de un método:

Dentro de `store()`, por ejemplo

```
if (! $request->user()->tokenCan('tarejas:crear')) {
    abort(403, 'Este token no puede crear tareas.');
```

Revocación de tokens

Objetivo	Código
Revocar el token actual (logout)	<code>\$request->user()->currentAccessToken()->delete();</code>
Cerrar sesión en todos los dispositivos	<code>\$request->user()->tokens()->delete();</code>
Revocar un token específico por id	<code>\$request->user()->tokens()->where('id', \$id)->delete();</code>

EXPIRACIÓN DE TOKENS

Por defecto los tokens de Sanctum no expiran. Para producción, define una expiración en `config/sanctum.php` (clave `expiration`, en minutos).

CHECKPOINT

Entiendes cómo asignar habilidades con `createToken()`, verificarlas con `tokenCan()` y revocar tokens de forma individual o masiva.

Actividad 7 · Probar el flujo completo

Probaremos el ciclo de extremo a extremo. Recuerda enviar siempre Accept: application/json.

1. Registrar un usuario: **POST** /api/register

cURL

```
curl -X POST http://127.0.0.1:8000/api/register \
-H "Accept: application/json" \
-d "name=Ana" \
-d "email=ana@uatf.edu.bo" \
-d "password=secret123" \
-d "password_confirmation=secret123" \
-d "device_name=postman"
```

La respuesta (201) incluye el token. Cópialo para las siguientes peticiones.

2. Crear una tarea autenticada: **POST** /api/tareas

cURL

```
curl -X POST http://127.0.0.1:8000/api/tareas \
-H "Accept: application/json" \
-H "Authorization: Bearer 1|TU_TOKEN_AQUI" \
-d "titulo=Preparar laboratorio" \
-d "vence_el=2026-07-01"
```

3. Listar mis tareas: **GET** /api/tareas

cURL

```
curl http://127.0.0.1:8000/api/tareas \
-H "Accept: application/json" \
-H "Authorization: Bearer 1|TU_TOKEN_AQUI"
```

4. Probar el rechazo: repite la petición anterior SIN la cabecera Authorization → debe responder 401 Unauthenticated.
5. Cerrar sesión: **POST** /api/logout. Tras esto, reutilizar el mismo token debe devolver 401.

SUGERENCIA EN POSTMAN

Guarda el token en una variable de entorno (p. ej. {{token}}) y usa la pestaña Authorization del request como «Bearer Token». Así no lo copias a mano en cada petición.

CHECKPOINT

Completaste el ciclo register → token → crear/listar tareas → 401 sin token → logout → 401 con token revocado. ¡La API está asegurada!

4. Rúbrica de Evaluación

La práctica se evalúa sobre 100 puntos.

Criterio	Descripción	Pts
Activación de Sanctum	Trait HasApiTokens en User; tabla personal_access_tokens operativa.	10
Pertenencia del recurso	Migración de user_id; relaciones belongsTo / hasMany; refactor aplicado.	15
Endpoints de autenticación	register, login, logout y user funcionan y emiten/revocan tokens.	20
Rutas protegidas	auth:sanctum aplicado; rutas públicas y privadas bien separadas.	15
Control de propietario	Acceso acotado por usuario; intento ajeno devuelve 403.	20
Habilidades y revocación	Uso correcto de abilities/tokenCan() y revocación de tokens.	10
Pruebas y evidencias	Flujo completo probado en Postman/cURL con capturas (incluido el 401).	10

TOTAL 100

5. Entregables

- Repositorio Git del proyecto api-tareas (continuación de la GL10) con commits por actividad.
- README.md actualizado con los endpoints de autenticación y el esquema de seguridad.
- Colección de Postman exportada (.json) con el flujo completo de autenticación.
- Documento PDF con las capturas, incluyendo la prueba de error 401 (sin token) y 403 (tarea ajena).

ETIQUETA DE ENTREGA (GIT TAG)

Al finalizar, marca esta versión como **v2.0** (continúa el repositorio de la GL10):

Terminal

```
git add .
git commit -m "GL11: seguridad de la API con Laravel Sanctum"
git tag v2.0
git push origin main --tags
```